

Ejercicio Demostrativo

Nombre del Auxiliar

Tiempo estimado: 30 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Amabilidad	Se evidencia en cómo el equipo comparte imágenes, reporta errores en contenedores y pide ayuda con configuraciones de Docker Compose. La forma en que se hace una solicitud técnica impacta directamente la disposición del compañero a colaborar. Un equipo amable construye la confianza necesaria para que el trabajo DevOps funcione fluidamente.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Analiza los conceptos fundamentales de Docker para el empaquetado y despliegue de aplicaciones en entornos reproducibles e independientes del sistema operativo.
Competencia Específica	Crea un Dockerfile para containerizar una API REST, construye y ejecuta la imagen, orquesta múltiples servicios con Docker Compose y sube la imagen a un Container Registry, completando el flujo de containerización requerido en el proyecto.

2. Palabras Clave

- **Dockerfile:** Archivo de texto con instrucciones para construir una imagen Docker de forma automatizada y reproducible.
- **Imagen Docker:** Plantilla inmutable que contiene el código, las dependencias y la configuración necesaria para ejecutar un contenedor.
- **Docker Compose:** Herramienta para definir y ejecutar aplicaciones multi-contenedor

mediante un archivo YAML.

- **Container Registry:** Servicio centralizado para almacenar, versionar y distribuir imágenes Docker (Docker Hub, GHCR, AWS ECR).
- **Multi-stage build:** Técnica de construcción de imágenes que usa múltiples etapas para producir una imagen final más pequeña y optimizada.

3. Resumen

Este ejercicio demostrativo guía al estudiante a través del flujo completo de containerización con Docker: partiendo de una API REST funcional en el *host*, se crea un Dockerfile para empaquetarla, se demuestra la diferencia entre *builds* de una etapa y multi-etapa, se orquesta la API junto con una base de datos usando Docker Compose y finalmente se sube la imagen al Container Registry. Este flujo es exactamente el requerido para el Corto 4 y la entrega final del proyecto.

4. Contenido

Descripción del ejercicio

Se trabaja sobre una API REST sencilla (puede ser la del proyecto del curso) que tiene dependencias de base de datos. El ejercicio cubre desde ejecutar la aplicación en el *host* hasta tenerla corriendo completamente en contenedores con todos sus servicios orquestados, y la imagen publicada en un registry accesible para cualquier miembro del equipo.

Objetivo de la demostración

Demostrar el flujo completo de containerización de una aplicación real con Docker, con el fin de que el estudiante comprenda cada etapa del proceso y pueda replicarlo de forma autónoma en su proyecto del curso.

Recursos necesarios

- Computadora con Docker Desktop instalado y en ejecución.
- Cuenta en Docker Hub o GitHub (para el Container Registry).
- Código de la API REST base proporcionado por el auxiliar o del proyecto del curso.

- Editor VS Code con extensión Docker instalada.
- Terminal integrada para ejecutar comandos.

Desarrollo paso a paso

Paso 1 — Demostrar la aplicación ejecutándose en el *host* (5 min)

El auxiliar ejecuta la API directamente en el host con `node app.js` o `python app.py` y realiza una petición de prueba para mostrar que funciona. Luego explica el problema: "Este código funciona en mi máquina, pero puede no funcionar en la tuya si tienes una versión diferente de Node, Python o las librerías." Este es el problema que Docker resuelve.

Paso 2 — Introducción a conceptos clave de Docker (5 min)

Se explican brevemente los conceptos fundamentales con el apoyo visual del proyector:

- **Imagen vs Contenedor:** la imagen es el plano, el contenedor es la casa construida a partir de ese plano.
- **Docker Hub:** el repositorio público de imágenes, equivalente a GitHub para código.
- **Capas:** cada instrucción del Dockerfile genera una capa; Docker reutiliza las capas sin cambios para acelerar las construcciones.

Paso 3 — Crear el Dockerfile

El auxiliar escribe el Dockerfile paso a paso, explicando cada instrucción:

None

```
# Imagen base oficial
FROM node:20-alpine

# Directorio de trabajo dentro del contenedor
WORKDIR /app
```

```
# Copiar solo los archivos de dependencias primero (aprovecha
  caché de capas)

COPY package*.json ./

# Instalar dependencias

RUN npm install --only=production

# Copiar el resto del código

COPY . .

# Puerto que expone la aplicación

EXPOSE 3000

# Comando de inicio

CMD ["node", "app.js"]
```

Se construye la imagen: `docker build -t mi-api:1.0.0 .`

Se ejecuta el contenedor: `docker run -p 3000:3000 mi-api:1.0.0`

Se verifica que la API responde igual que en el *host*.

Paso 4 — Demostrar *builds* multi-etapa y archivo `.dockerignore`

El auxiliar muestra la diferencia entre una imagen de una sola etapa (que incluye herramientas de desarrollo innecesarias en producción) y una imagen multi-etapa optimizada:

None

```
# Etapa de construcción

FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build


# Etapa de producción (imagen final más pequeña)

FROM node:20-alpine AS production

WORKDIR /app

COPY --from=builder /app/dist ./dist

COPY --from=builder /app/node_modules ./node_modules

EXPOSE 3000

CMD ["node", "dist/app.js"]
```

Se comparan los tamaños de ambas imágenes con `docker images`. También se muestra el archivo `.dockerignore` para excluir `node_modules`, `.env` y archivos de prueba de la imagen final.

Paso 5 — Crear y ejecutar con Docker Compose

None

```
version: '3.8'

services:
```

```
api:

  build: .

  ports:
    - "3000:3000"

  environment:
    - DB_HOST=db
    - DB_PORT=5432
    - DB_NAME=mydb

  depends_on:
    - db

db:

  image: postgres:15-alpine

  environment:
    - POSTGRES_DB=mydb
    - POSTGRES_USER=admin
    - POSTGRES_PASSWORD=secret

  volumes:
    - postgres_data:/var/lib/postgresql/data

volumes:

  postgres_data:
```

Se ejecuta con `docker compose up` y se verifica que ambos servicios estén en funcionamiento. Se demuestra `docker compose down` para detener todo limpiamente.

Paso 6 — Subir la imagen al Container Registry (10 min)

El auxiliar demuestra el flujo completo de publicación en Docker Hub:

None

```
# Iniciar sesión
docker login

# Etiquetar con el nombre de usuario
docker tag mi-api:1.0.0 usuario/mi-api:1.0.0

# Subir al registry
docker push usuario/mi-api:1.0.0

# Desde otro equipo o servidor:
docker pull usuario/mi-api:1.0.0
```

Se accede a Docker Hub en el navegador para mostrar la imagen publicada con sus metadatos y el *tag* de versión.

Observaciones importantes

- Docker Desktop debe estar ejecutándose antes de la sesión; los estudiantes deben verificarlo al inicio.
- El archivo `.env` nunca debe copiarse a la imagen Docker; las variables de entorno sensibles se pasan en tiempo de ejecución.
- `docker compose up` y `docker-compose up` son comandos diferentes (V2 vs V1 de

Compose); usar la versión V2 integrada en Docker Desktop.

- Si la imagen ya existe localmente, `docker pull` no la descarga de nuevo a menos que el *tag* haya cambiado; usar *tags* de versión semántica evita confusiones.
- Antes de hacer *push* al registry, verificar que la imagen funciona correctamente en local.

Tabla 1 — Comandos Clave de Docker Demostrados

Comando	Descripción	Momento de uso
<code>docker build -t nombre:tag .</code>	Construye una imagen desde el Dockerfile	Al crear o actualizar la imagen
<code>docker run -p 3000:3000 nombre:tag</code>	Ejecuta un contenedor exponiendo el puerto	Para probar la imagen localmente
<code>docker images</code>	Lista las imágenes disponibles localmente	Para comparar tamaños de imágenes
<code>docker compose up</code>	Levanta todos los servicios definidos en <code>docker-compose.yml</code>	Para iniciar el entorno completo
<code>docker compose down</code>	Detiene y elimina los contenedores del Compose	Para limpiar el entorno
<code>docker push usuario/nombre:tag</code>	Sube la imagen al Container Registry	Para compartir o desplegar
<code>docker pull usuario/nombre:tag</code>	Descarga la imagen desde el Registry	Para obtener la versión publicada

Resultado esperado

Al finalizar la demostración, el estudiante debe ser capaz de crear un Dockerfile para containerizar su aplicación, construir y ejecutar la imagen localmente, definir un `docker-compose.yml` que orqueste la API con su base de datos y subir la imagen a Docker Hub o GHCR con el tag de versión correcto. Este resultado es directamente requerido en el Corto 4 y en la entrega final del proyecto.

Aplicación práctica

El flujo demostrado en esta sesión es el estándar de la industria para el despliegue de aplicaciones modernas. Dominar Docker permite que el proyecto del curso sea desplegable en cualquier servidor sin problemas de compatibilidad, y sienta las bases para prácticas más avanzadas como integración continua con GitHub Actions, orquestación con Kubernetes y despliegues en la nube.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

- **Dificultades encontradas:** Los estudiantes suelen confundir imagen con contenedor al inicio de la sesión. También es frecuente que el comando `docker compose up` falle porque el servicio de base de datos aún no está listo cuando la API intenta conectarse; es importante mostrar el uso de `depends_on` y, si aplica, un *script* de espera.
- **Recomendaciones para futuras sesiones:** Preparar el repositorio base con antelación y verificar que `docker build` funciona sin errores antes de la sesión. Mostrar primero el problema ("funciona en mi máquina") antes de introducir Docker, para que los estudiantes entiendan el valor de la herramienta antes de aprender los comandos.
- **Puntos de mayor dificultad para el estudiante:** Entender el mapeo de puertos (`-p hostPort:containerPort`) y por qué se necesita. También el manejo de variables de entorno en Docker Compose sin exponer credenciales en el archivo YAML.

6. Referencias

1. Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com>
2. Docker Inc. (2024). *Docker Compose Overview*. <https://docs.docker.com/compose/>
3. GitHub Docs. (2024). *Working with the Container Registry (GHCR)*. <https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry>